

Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Desarrollo de Aplicaciones Web	Unidad 4	Tema: Contenedorización con Docker
	Semana 14	Tutor: Ing. Victoria Castro

Introducción

Contenedores de Software

En la ingeniería de software moderna, el despliegue de aplicaciones enfrentaba históricamente el reto de la inconsistencia de entornos (diferencias entre el desarrollo, pruebas y producción). La **contenedorización** es una tecnología de virtualización a nivel de sistema operativo que permite empaquetar una aplicación junto con todas sus dependencias, bibliotecas y archivos de configuración en una única unidad aislada llamada **contenedor**.

A diferencia de las Máquinas Virtuales (VM), los contenedores no incluyen un sistema operativo invitado completo; en su lugar, comparten el kernel del sistema operativo del host. Esto los hace extremadamente ligeros, rápidos de iniciar y eficientes en el uso de recursos.

Sección I: Ecosistema Docker: Componentes Esenciales

Para dominar Docker, es fundamental entender la diferencia y relación entre sus cuatro pilares conceptuales:

A. Imágenes (Images)

Según la documentación oficial de Docker, una imagen es un paquete ejecutable, ligero y autónomo que incluye todo lo necesario para ejecutar una aplicación: el código, el entorno de ejecución (runtime), las herramientas del sistema, las bibliotecas y las configuraciones.

- **Características clave:** Son inmutables (no cambian una vez creadas) y están compuestas por capas de solo lectura. Se almacenan en registros como *Docker Hub*.
- **Analogía en Ingeniería:** Es el equivalente a una Clase en la Programación Orientada a Objetos (POO) o a un plano arquitectónico.

B. Contenedores (Containers)

Un contenedor es una instancia de ejecución de una imagen. Es el proceso vivo y aislado que corre en la máquina host.

- **Características clave:** Tienen un ciclo de vida (se crean, inician, detienen y eliminan). Añaden una delgada capa de escritura/lectura sobre la imagen base. Si el contenedor se destruye, cualquier dato guardado en esa capa de escritura se pierde (naturaleza efímera).
- **Analogía en Ingeniería:** Es el equivalente a un Objeto instanciado en POO.

C. Puertos (Ports)

Por defecto, los contenedores están completamente aislados del mundo exterior y de la máquina host. Para que una aplicación dentro de un contenedor (por ejemplo, un servidor web) sea accesible, se utiliza el mapeo o publicación de puertos (-p o --publish).

- Esto vincula un puerto de la máquina host con un puerto interno del contenedor, creando un puente de comunicación de red.

D. Volúmenes (Volumes)

Dado que los contenedores son efímeros, Docker introduce los Volúmenes como el mecanismo recomendado para la persistencia de datos. Un volumen es un directorio gestionado directamente por Docker en el sistema de archivos del host, aislado de la estructura del contenedor. Permite que los datos sobrevivan incluso si el contenedor se destruye o se actualiza.

Ejemplo Práctico: Conectando Puertos y Volúmenes

Imaginemos que vamos a desplegar una base de datos **PostgreSQL** usando Docker. Necesitamos que los datos no se borren y que los desarrolladores puedan conectarse a ella desde sus herramientas locales (como DBeaver o pgAdmin).

El comando en la terminal se vería así:

Bash

```
docker run -d \  
  --name mi_base_datos \  
  -p 5432:5432 \  
  -v mi_data_postgres:/var/lib/postgresql/data \  
  postgres:15
```

Explicación de la arquitectura del comando:

1. **-p 5432:5432 (Puertos):** El primer 5432 es el puerto de la *máquina host*; el segundo 5432 es el puerto interno del *contenedor*. Cualquier petición que llegue a la IP de la laptop/servidor en el puerto 5432 será redirigida automáticamente al PostgreSQL del contenedor.
2. **-v mi_data_postgres:/var/lib/postgresql/data (Volúmenes):** Crea un volumen llamado *mi_data_postgres* en el host y lo monta en la ruta interna donde PostgreSQL guarda la base de datos de manera nativa. Si el contenedor se elimina con `docker rm`, los datos siguen a salvo en el volumen y se pueden conectar a un contenedor nuevo.

Sección 2: Dockerfile y el Empaquetado de Aplicaciones

¿Qué es un Dockerfile?

Un **Dockerfile** es un documento de texto plano que contiene todas las instrucciones, comandos y argumentos necesarios que un usuario ejecutaría de forma manual en la terminal para ensamblar una imagen de Docker.

Docker lee este archivo secuencialmente de arriba hacia abajo para ejecutar las instrucciones y construir (*build*) la imagen automatizadamente.

Cada instrucción en un Dockerfile suele generar una nueva capa en la imagen:

- **FROM:** Define la imagen base (ej. un sistema operativo o un entorno como Node.js o Java).
- **COPY / ADD:** Copia archivos del proyecto local al contenedor.
- **RUN:** Ejecuta comandos de configuración (instalar dependencias, compilar).
- **CMD / ENTRYPOINT:** Define el comando por defecto que se ejecutará cuando el contenedor inicie.

Sección 3: ¿Para qué sirve el empaquetado de aplicaciones en la Ingeniería de Software?

En el ciclo de vida del desarrollo de software (SDLC), el empaquetado mediante contenedores cumple objetivos estratégicos vitales:

1. **Garantía de Idoneidad Ambiental (Idempotencia):** Asegura que el artefacto generado se comporte exactamente igual en la máquina del desarrollador, en el servidor de integración continua (CI/CD) y en el clúster de producción (Kubernetes, AWS, etc.).
2. **Aislamiento de Dependencias:** Permite que en el mismo servidor coexistan una aplicación que requiere Java 8 y otra que requiere Java 21, sin que ocurran conflictos de variables de entorno globales.
3. **Trazabilidad y Control de Versiones:** Al ser el Dockerfile puro código, se almacena en Git. El proceso de infraestructura se vuelve auditable y versionable (*Infrastructure as Code*).

4. **Escalabilidad Horizontal Eficiente:** Al estar empaquetada la aplicación de forma compacta, los sistemas de orquestación pueden levantar 10 o 100 instancias idénticas en segundos ante picos de tráfico de usuarios.

Bibliografía y Fuentes Oficiales

Docker Inc. (2026). Docker Documentation: Product manuals and reference. Recuperado de docs.docker.com

Turnbull, J. (2014). The Docker Book: Containerization is the new virtualization.

Burns, B. (2019). Designing Distributed Systems: Patterns and Paradigms for Scalable, Reliable Services. O'Reilly Media.

